Verifiably Correct Implementations of Basic Data Structures in Solidity

Bachelor's Thesis by

Jonas Wille

at the KIT Department of Informatics
Institute of Information Security and Dependability (KASTEL)

Reviewer: Prof. Dr. Bernhard Beckert
Advisor: Jonas Schiffl, M. Sc.

15 January 2023 – 15 May 2023

Karlsruher Institut für Technologie
KIT-Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

I hereby declare that the work presented in this thesis is entirely my own. I confirm that I specified all employed auxiliary resources and clearly acknowledged anything taken verbatim or with changes from other sources. I further declare that I prepared this thesis in accordance with the rules for safeguarding good scientific practice at Karlsruhe Institute of Technology (KIT).

Karlsruhe, 15 May 2023

. . . . . .

(Jonas Wille)

Abstract

Smart contracts form the backbone of decentralised applications. Decentralised applications often feature substantial financial components and thereby attract many bad actors seeking to exploit vulnerabilities in the underlying smart contract. Vulnerabilities can allow an attacker to exploit contract behaviour in a way that might not have been intended by the creator. In the realm of the Ethereum blockchain, this issue is exacerbated by the fact that the bytecode and often even the source code of the smart contracts are publicly visible. This enables anyone to analyse the workings of the smart contract and discover potential attack vectors. As a result of these factors, smart contracts must meet high demands for security and resilience.

This thesis contributes to secure and functionally correct smart contracts by exploring the prospect of implementing basic data structures in Solidity, a programming language used to develop smart contracts for the Ethereum blockchain. The primary focus is to provide data structures that cover some basic use cases for smart contract development and ensure the implementations' correctness using formal verification. The chosen data structures are lists, sets, iterable mappings and caller specific mappings. We provide insights into how and where these data structures can be used and discuss the challenges encountered during the implementation and verification. The results of this thesis contribute to making smart contracts more reliable and secure, by presenting four functionally verified primitive data structures. Functionally verified primitives are advantageous as they act as basic building blocks, featuring explicitly known and guaranteed behaviour, which can be used to build contracts with high degrees of security.

Verifying the data structures with the basic Solidity data types was mostly a success. Providing implementations for more user-defined types was also explored and implemented, however, no verification could be supplied. The syntactical limitations of the used tool and the lack of support for newer Solidity versions presented significant difficulties in this regard.

Zusammenfassung

Mit der zunehmenden Verbreitung von Blockchain-basierten Diensten in Form von dezentralen Anwendungen müssen Smart Contracts, die deren Rückgrat bilden, hohe Anforderungen an Sicherheit und Robustheit erfüllen. Dezentrale Anwendungen beinhalten oft umfangreiche finanzielle Komponenten und ziehen daher viele böswillige Akteure an, die versuchen, Schwachstellen in den zugrunde liegenden Smart Contracts auszunutzen. Schwachstellen können es einem Angreifer ermöglichen, Funktionen auf eine Art und Weise zu nutzen, die vom Ersteller möglicherweise nicht beabsichtigt war. Im Bereich der Ethereum-Blockchain wird dieses Problem durch die Tatsache verschärft, dass der Bytecode und oft sogar der Quellcode der Smart Contracts öffentlich einsehbar sind. Dies ermöglicht es jedem, die Funktionsweise des Smart Contracts zu analysieren und potenzielle Angriffsvektoren zu entdecken.

Diese Arbeit untersucht die Möglichkeit, grundlegende Datenstrukturen in Solidity zu implementieren, einer Programmiersprache, die zur Entwicklung von Smart Contracts für die Ethereum-Blockchain verwendet wird. Der primäre Fokus liegt auf der Bereitstellung von Datenstrukturen, die einige grundlegende Anwendungsfälle für die Entwicklung von Smart Contracts abdecken und die Korrektheit der Implementierungen durch formale Verifikation sicherstellen. Die gewählten Datenstrukturen sind Listen, Mengen, iterierbare Mappings und Caller Specific Mappings. Wir geben Einblicke, wie und wo diese Datenstrukturen eingesetzt werden können und diskutieren die Herausforderungen, die bei der Implementierung und Verifikation aufgetreten sind. Die Ergebnisse dieser Arbeit tragen dazu bei, Smart Contracts zuverlässiger und sicherer zu machen, indem vier funktional verifizierte primitive Datenstrukturen bereitgestellt werden. Funktional verifizierte Primitive sind vorteilhaft, da sie als Grundbausteine mit explizit bekanntem und garantiertem Verhalten fungieren, die zum Aufbau von Smart Contracts mit hohem Sicherheitsgrad verwendet werden können.

Die Verifizierung der Datenstrukturen mit den Solidity Basisdatentypen war größtenteils ein Erfolg. Die Bereitstellung von Implementierungen für weitere benutzerdefinierte Typen wurde ebenfalls untersucht, jedoch konnte keine Verifizierung geliefert werden. Die syntaktischen Beschränkungen des verwendeten Werkzeugs und die fehlende Unterstützung für neuere Solidity-Versionen stellten in dieser Hinsicht erhebliche Schwierigkeiten dar.

Contents

1 Introduction **1**

2 Preliminaries **3**
2.1 Solidity . . . 3
2.2 Formal Specification and Verification . . . 4

3 Formal Definitions **5**
3.1 Lists . . . 5
3.1.1 List Operations . . . 5
3.2 Sets . . . 8
3.2.1 Set Operations . . . 9
3.3 Iterable Mappings . . . 10
3.3.1 Iterable Mapping Operations . . . 10
3.4 Caller Specific Mappings . . . 12
3.4.1 Caller Specific Mapping Operations . . . 13
3.5 Iterator . . . 14
3.5.1 Iterator Operations . . . 14

4 Implementation in Solidity **15**
4.1 General . . . 15
4.1.1 Approach . . . 15
4.1.2 solc-verify . . . 16
4.2 Challenges . . . 18
4.2.1 Generics in Solidity . . . 18
4.2.2 Libraries and Contracts in Solidity . . . 20
4.2.3 The Iterator Design . . . 21
4.2.4 Deviations of the Concrete Implementations from the Abstract Definitions . . . 22
4.2.5 Functions in the Specification . . . 26
4.3 Usage of the Data Structures in Solidity Source Code . . . 27

5 Evaluation **29**
5.1 Limitations . . . 29
5.2 Timing the Verification . . . 30
5.3 Example Use Cases of the Data Structures . . . 31

6 Related Work **33**

7 Conclusion **35**

Bibliography **37**

A Appendix **39**
A.1 Data Locations in Solidity 39
A.2 Timing Tables 40

1 Introduction

Solidity is a programming language that is used to develop Smart Contracts for the Ethereum (Buterin, 2014) platform. A smart contract is an entity that lives on the blockchain and can provide services there. The services often include managing valuable items or acting as a representation and execution of an agreement between different parties. Ensuring the correctness of smart contracts from the start and aligning them with the functional expectations of those responsible is especially crucial, given that their bytecode is publicly visible and they cannot be altered after deployment on the blockchain. If sufficient security is not achieved the contract can be vulnerable to devastating exploits or unexpected behaviour. Such was the case with the DAO smart contract where an attacker was able to exploit vulnerabilities in the contract code to steal over $50M in Ether (Mehar et al., 2019). Unfortunately, there are many vulnerabilities to watch out for (Atzei, Bartoletti, and Cimoli, 2017) and incidents of this scale are not isolated either and have occurred sporadically since the inception of Ethereum (Sayeed, Marco-Gisbert, and Caira, 2020).

There exist many tools (Salzer and Di Angelo, 2019), principles (Wohrer and Zdun, 2018) and strategies developers can fall back on to increase the security of their smart contracts. This thesis contributes in this regard, by supplying Solidity implementations of list, set, iterable mapping and caller specific mapping data structures and verifying them functionally using the solc-verify tool (Hajdu and Jovanović, 2020). Lists and sets were chosen, as the only dynamically sized data structures available directly in Solidity are unbounded arrays. While unbounded arrays are powerful, they can present difficulties when dealing with element insertion and removal. Iterable and caller specific mappings are alterations to the mapping type which exists in Solidity. A normal Solidity mapping is not iterable, that is information about keys is not queryable. Iterable mappings address this shortcoming by providing the same functionality as a regular mapping, but additionally being iterable. Caller specific mappings are specialised access control integrated mappings for the use case of storing user specific information. They prevent users from accessing values in the mapping that is not associated with them.

These implementations can be used in future Solidity smart contracts but also act as a proof of concept that data structures can be implemented in Solidity and verified functionally using existing tools. Functionally verified primitives provide value to the Solidity ecosystem. A developer needing specific functionality that may not (yet) exist explicitly in Solidity must not implement it themselves, thereby introducing unnecessary complexity and possible errors to their contract. Instead, this developer can fall back on existing functionally verified primitives and can also immediately see from the specification which guarantees are made by the verification. Through this, ideally, the development of Solidity Smart Contracts can become a little more robust and secure.

The Preliminaries chapter briefly provides background information on Solidity and formal verification. Formal definitions and explanations of the individual data structures can be found in chapter 3. The general approach and details of the concrete implementations can be found in chapter 4. Chapter 5 reflects the results from chapter 4. Chapter 6 contains related work. Finally, chapter 7 holds the conclusion and an outlook on future work.

2 Preliminaries

2.1 Solidity

Solidity is a domain-specific programming language to author smart contracts for the Ethereum blockchain. Smart contracts are programs which are stored on the blockchain. In abstract terms, smart contracts automatically enforce the terms of an agreement between multiple parties in a decentralised manner. The *Solidity Documentation* n.d. describes the language itself as follows, "Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs which govern the behaviour of accounts within the Ethereum state". Through smart contracts on the Ethereum blockchain functionality such as voting, crowdfunding, auctioning, and digital assets management can be implemented.

Solidity is currently being developed by the Ethereum Foundation. Its compiler is open source, meaning anyone can view its source code and suggest changes. It was influenced by C++, Javascript and Python, hence its syntax bears some resemblance to these languages. However, the language is also substantially different not only due to its unique execution environment but also because it features blockchain-specific functionality. As Solidity is a relatively new programming language and the blockchain domain has its own distinctive characteristics, writing Solidity code can sometimes feel unfamiliar. A few key concepts that one might expect to be available, such as generics, do not exist (yet) in Solidity. This topic will come up again in later chapters.

For this thesis, three Solidity constructs are important: contracts, libraries and structs. A contract in Solidity is the analogue for classes in object-oriented programming languages. In simplified terms, a contract can have contract-level variables (these are like object attributes) and functions. Supported data types for variables are booleans, integers, addresses, fixed and dynamic length bytes, strings, enums, structs, arrays and mappings.[1] Depending on their visibility, functions can be called from the outside once the contract is deployed and can then modify the internal state of the contract. Libraries are similar to contracts except they cannot hold any internal state themselves, they can only define methods and constants that can then be used in a contract. Lastly, structs are analogous to the struct concept from the C programming language. They are used to define a group of variables and can act as basic object templates.

While contracts are comparable to classes, they are not instantiated the way a class would be in an object-oriented programming language. Every instantiation of a contract is unique and will be deployed to the blockchain as a separate entity. To make up for this behaviour developers can use structs to define and instantiate simple object structures for their use case resulting in better modelling and reduction of complexity. A struct initialised

[1] `https://docs.soliditylang.org/en/v0.7.0/types.html`

in a contract will be part of that contract, a contract A being initialised in contract B will mean A and B are both deployed, and B contains a reference to the address of A.

2.2 Formal Specification and Verification

An important aspect of this thesis is the formal verification of source code. Formally verifying something is the process of proving whether a system (in this case source code) satisfies a specification. The specification details the intended behaviour and contains conditions that the system meets or does not meet. The job of the formal verification is to determine if the system meets its specification. The specification for the source code, in our case, is written in the form of function contracts. A function contract consists of post-, pre- and frame conditions. A function fulfils its contract when the following relationship is true: the function always terminates in a state that fulfils the postconditions given that it was called in a state that fulfils the preconditions. The frame conditions specify which elements can be altered during the course of the function call. The responsibility of the verification tool is then to prove (or disprove) that a function fulfils its contract and frame conditions.

In addition to function contracts the concept of invariants is also important. For our purposes, we can differentiate between two types of invariants: class invariants and loop invariants. A class invariant is used to assert system requirements. A value always being within a certain range, or a sum of values always being greater than zero are some simple examples. A class invariant means the conditions specified must hold at every point in time when an instantiation of the class is accessible from the outside, this means especially before and after every method call. While executing a method, the class invariant may be temporarily violated, provided that it is restored before the method finishes. A loop invariant on the other hand is an auxiliary piece of specification that is required for the proof of other pieces of specification. A loop invariant is a statement that is true before and after every iteration of a loop, while the control flow is in the loop itself the invariant can be momentarily broken.

Our tool of choice to verify Solidity source code is solc-verify Hajdu and Jovanović, 2020. Solc-verify is an extended version of the Solidity compiler that introduces a specification language and a verifier. Using the specification language, we can add constructs such as pre- and postconditions (and others) to the source code and call the tool on the file to check for inconsistencies between the specification and the source code. For examples and further information about solc-verify reference subsection 4.1.2.

3 Formal Definitions

Within this chapter's subsections, we formally define the data structures we are interested in. Specifically, a general description and individual methods are listed for every data structure. The functionality of the methods is described in natural language, additionally they are supplemented with pre- and postconditions that constrain and describe their behaviour precisely. The later implementation will be based on the pre- and postconditions and method signatures.

3.1 Lists

A list in our understanding is a dynamic data storage structure. A specific list stores elements of exactly one data type. The list has a length, representing the number of elements currently stored by it. Elements can be added and removed from the list, when elements are added to the list they are virtually associated with an index. The index of an individual element must be a natural number from zero to the length of the list minus one. The index at a given time of an element represents its position in the list, and can be used to retrieve this element from the list. In the lifespan of a list, the index of an individual element may change, depending on add and remove operations. Through the index the list maintains information about the insertion order of elements. Elements added earlier will have a lower index than elements added later, assuming they aren't removed in the meantime. The element with the highest index in the list is considered the last element, all indices between zero and the highest index are occupied by other elements, and every index refers to exactly one element. Besides direct element access, elements can also be queried by using an iterator's `next` and `hasNext` functions.

Equality of lists We consider two lists to be equal if and only if they have the same length and contain the same elements in the same order, meaning that each pair of elements from the lists with the same index are also equal. By this definition, lists of length zero are equal.

3.1.1 List Operations

For the following section, *List* denotes an instantiation of our data structure. List.length refers to the length of the list List (that is the number of elements that it currently stores). List[i] refers to the element at the index i of the list List, assuming i is a valid index for List (that is between zero and List.length − 1). List[i:n] refers to a list comprised of the elements at index i (inclusive) to index n (exclusive) of the list List, where i and n are valid indices of List and $i \leq n$. old(*exp*) denotes the value of the expression *exp* as it would

have been before an operation was called on the list. \result denotes the return value of a particular operation.

Constructor The constructor returns a new instantiation of an empty list data structure.

Size The Size operation returns the current length of the list, that is the number of elements it holds. It does not alter the list in any way.

Postconditions:

- $List = \text{old}(List)$
- $\backslash\text{result} = List.\text{length}$

Get (index i) Get returns the element from the list which is at index i, where index i is a valid index of the list. Get is simply an explicit operation returning $List[i]$. As with size, the operation does not alter the list in any way:

Preconditions:

- $0 \leq i < List.\text{size}()$

Postconditions:

- $List = \text{old}(List)$
- $\backslash\text{result} = List[i]$

Contains (element e) Contains checks if the list contains a certain element. If so, it shall return True, otherwise False. Once again the list Contains is called on is not altered in any way.

Postconditions:

- $List = \text{old}(List)$
- $\exists i.(0 \leq i \wedge i < List.\text{size}() \wedge List[i] = e) \leftrightarrow \backslash\text{result} = \text{True}$

IndexOf (element e) IndexOf returns the first index of an element in the list which is equal to e. Should there be no such element in the list, the operation signals this by returning -1 as an index. The operation IndexOf does not alter the list in any way.

Postconditions:

- $List = \text{old}(List)$
- $List.\text{contains}(e) \leftrightarrow \backslash\text{result} \neq -1$
- $\backslash\text{result} \neq -1 \rightarrow List[\backslash\text{result}] = e$
- $\backslash\text{result} \neq -1 \rightarrow \forall i.((i \leq 0 \wedge\ i < \backslash\text{result}) \rightarrow \text{array}[i] \neq e)$
- $\backslash\text{result} = -1 \rightarrow \forall i.((i \leq 0 \wedge\ i < List.\text{size}()) \rightarrow \text{array}[i] \neq e)$

Add (element e) Add inserts a new element e to the end of the list, increasing the list length by one. The rest of the elements as well as their indices are unchanged.

Postconditions:

- $List[List.\text{size}() - 1] = e$
- $List[0 : List.\text{size}() - 1] = \text{old}(List[0 : List.\text{size}()])$
- $List.\text{size}() = \text{old}(List.\text{size}()) + 1$

Remove (index i) Remove removes the element from the list which is at index i, where index i is a valid index of the list. The list length has now decreased by one. All other elements in the list with indices less than that of i remain unchanged and they keep their index. Elements with indices greater than i also remain unchanged, however, their index decreases by one. Essentially they move up the list as a response to an element in front of them being removed. Besides the removed element, the order is maintained.

Preconditions:

- $0 \leq i < List.\text{size}()$

Postconditions:

- $List[0 : i] = \text{old}(List[0 : i])$
- $List[i : List.\text{size}()] = \text{old}(List[i + 1 : List.\text{size}()])$
- \result = $\text{old}(List[i])$

Remove (element e) Remove removes the first occurrence of an element which is equal to e from the list, provided that such an element exists in the list. If this is the case, the list length has now decreased by one. All other elements in the list with indices less than that of the removed element remain unchanged and they keep their index. Elements with indices greater than the removed element also remain unchanged, however, their index decreases by one. Essentially they move up the list as a response to an element before them being removed. Besides the removed element, the order is maintained. If no element which is equal to e exists in the list, the list remains unchanged.

Postconditions:

- $\text{old}(List.\text{contains}(e)) \leftrightarrow$ \result = True
- \result = True $\rightarrow List.\text{size}() = \text{old}(List.\text{size}) - 1$
- \result = True $\rightarrow List[0 : \text{old}(List.\text{indexOf}(e))] = \text{old}(List[0 : List.\text{indexOf}(e)])$
- \result = True $\rightarrow List[\text{old}(List.\text{indexOf}(e)) : List.\text{size}()] = \text{old}(List[List.\text{indexOf}(e)+ 1 : List.\text{size}()]))$
- \result = False $\rightarrow List = \text{old}(List)$

IsEmpty IsEmpty returns whether the list contains no elements. If it contains no elements IsEmpty returns True, otherwise it returns False. Calling IsEmpty on a list does not alter it in any way.

Postconditions:

- $List = \text{old}(List)$
- \result = True $\leftrightarrow List.\text{size}() = 0$
- \result = False $\leftrightarrow List.\text{size}() > 0$

Set (index i, element e) Set overwrites the element at the index i in the list and replaces it with the element e. The index i must be a valid index for the list. Naturally, the Set operation can alter the element at index i, but all other elements at other indices are untouched.

Preconditions:

- $0 \leq i < List.\text{size}()$

Postconditions:

- $List[0 : i] = \text{old}(List[0 : i])$
- $List[i + 1 : List.\text{size}()] = \text{old}(List[i + 1 : List.\text{size}()])$
- $List[i] = e$

Clear Clears removes all elements that were stored in the list. The list will now be empty.

Postconditions:

- $List.\text{size}() = 0$
- $List = \text{old}(List)$

3.2 Sets

A set in our understanding, like a list, is a dynamic data storage structure, however, the key difference is that a set makes no guarantees about the preservation of the item order. A specific set stores elements of exactly one data type. The set has a length representing the number of items currently stored by it. Elements can be added and removed from the set. The elements of a set unlike those of a list, do not have an index. As mentioned above, a set will not make any guarantees about maintaining the insertion order of the elements. A newly added element may end up before or after an element that was inserted beforehand. While a set may not offer any direct element access it offers iterator-based access. Elements can be queried by using an iterator's `next` and `hasNext` functions.

Equality of sets We consider two sets A and B to be equal if they have the same size, and they contain the same elements, meaning set A contains a certain element if and only if set B contains that element. By this definition sets of length zero are equal.

3.2.1 Set Operations

For the following section *Set* denotes an instantiation of our set data structure. Set.length refers to the length of the set Set (that is the number of elements that it currently stores). old(*exp*) denotes the value of the expression *exp* as it would've been before an operation was called on the set. While the elements of a set do not have an index, we use Set[i] to refer to the element that would be assigned the index i as a result of iterating over the set Set. This choice was made to make the specification easier and to not have to work with iterators in the statements. \result denotes the return value of a particular operation.

Constructor The constructor returns a new instantiation of a set data structure that is empty.

Size Size returns the current length of the set, that is the number of elements it holds. It must return the current length of the set and otherwise does not alter it in any way.

Postconditions:

- $Set = \text{old}(Set)$
- $\backslash\text{result} = Set.\text{length}$

Contains (element e) Contains evaluates whether the set currently holds the element *e* in its storage. Should this be the case it returns True, otherwise it returns False. Contains does not alter the set in any way.

Postconditions:

- $Set = \text{old}(Set)$
- $\exists i.(0 \leq i \wedge i < Set.\text{size}() \wedge Set[i] = e) \leftrightarrow \backslash\text{result} = \text{True}$

Add (element e) Add attempts to inserts a new element *e* to the set. If the set already contained the element *e* the set remains unchanged and False is returned. If the set did not contain *e* before the Add operation, the set now contains all elements it had before the Add operation and additionally also the element e. In this case, True is returned and the size of the set increases by one.

Postconditions:

- $\text{old}(\overline{Set.\text{contains}(\ e)}) \leftrightarrow \backslash\text{result} = \text{True}$
- $\backslash\text{result} = \text{True} \rightarrow Set.\text{size}() = \text{old}(Set.\text{size}()) + 1$
- $\backslash\text{result} = \text{False} \rightarrow Set = \text{old}(Set)$

- $\forall k.(\text{old}(Set.\text{contains}(k)) \rightarrow Set.\text{contains}(k))$
- $Set.\text{contains}(e)$

Remove (element e) Remove removes e from the set, if the set does not contain e, the set remains unchanged. Should the set contain the element e, the set length is reduced by one. All the elements that were previously in the set and are not e are still in the set and are unchanged.

Postconditions:

- $\text{old}(Set.\text{contains}(e)) \leftrightarrow \backslash\text{result} = \text{True}$
- $\backslash\text{result} = \text{True} \rightarrow Set.\text{size}() = \text{old}(Set.\text{size}()) - 1$
- $\backslash\text{result} = \text{False} \rightarrow Set = \text{old}(Set)$
- $\forall k.((k \neq e \wedge \text{old}(Set.\text{contains}(k))) \rightarrow Set.\text{contains}(k))$
- $\overline{Set.\text{contains}(e)}$

3.3 Iterable Mappings

Solidity offers mappings as one of its primitive data types. Mappings are akin to hash tables and offer the ability to store and retrieve information based on a key, by mapping keys to values. One peculiarity of Solidity mappings is that on initialisation all keys are virtually set with standard values. In particular, retrieval of information on which keys are set and which are unset is not possible, as the concept of an unset key is not present. Because of this limitation, operations such as one to iterate over all set keys and perform an action on the values are not possible. This problem is what iterable mappings in the sense of this thesis aim to solve. Therefore, an iterable mapping can be understood as a data storage structure that enables the storage and retrieval of information based on keys. Each key is mapped to at most one value. Furthermore, the iterable mappings have a size representing the number of keys that are set in the iterable mapping. Just like with a regular mapping, new keys can be associated with values, the value associated with a specific key can be retrieved, and the values of already set keys can be overridden. Similar to sets and lists an iterable mapping must offer iterator-based element access, in this case over the keys, with the next and hasNext operators.

Equality of Iterable Mappings We consider two iterable mappings A and B to be equal if they have the same size, that is they map the same amount of keys and they map the same keys to the same values. Meaning iterable mapping A maps a certain key to a certain value if and only if iterable mapping B maps the same key to the same value.

3.3.1 Iterable Mapping Operations

For the following subsection *IMap* denotes an instantiation of our iterable mapping data structure. IMap.length refers to the length of the iterable mapping IMap, representing the

number of keys that are currently set. Once again as was the case with sets, the keys of an iterable mapping themselves do not have an index. However, we use IMap.keys[i] to refer to the key that would receive the index i as a result of iterating over the keys of an iterable mapping. The first key of the iteration would receive the index zero, the second key the index one, continued for all the keys currently mapped by the iterable mapping. IMap[k] refers to the value currently associated with the key k. \result denotes the return value of a particular operation.

Constructor The constructor returns a new instantiation of an iterable mapping data structure that is empty.

Get (key k) Get retrieves the value stored in the iterable mapping associated with the key k. Should the key not be present in the iterable mapping currently the operation returns a zero value, meaning it returns an instance of the return type that consists entirely of zero bytes. The Get operation does not alter the iterable mapping in any way.

Postconditions:

- $IMap = \text{old}\,(IMap)$
- \returns $= IMap[k]$

Size Size returns the current number of keys that are set in the iterable mapping, it does not alter the iterable mapping in any way.

Postconditions:

- $IMap = \text{old}\,(IMap)$
- \returns $= IMap.\text{keys.length}$

ContainsKey (key k) The operation ContainsKey returns True if the iterable mapping contains a mapping for the key k and False otherwise. The operation does not alter the iterable mapping in any way.

Postconditions:

- $IMap = \text{old}\,(IMap)$
- \returns $= \text{True} \rightarrow \exists i.(IMap.\text{keys}[i] = k)$
- \returns $= \text{False} \rightarrow \forall i.((0 \leq i \wedge i < IMap.\text{keys.length} \rightarrow IMap.\text{keys}[i] \neq k)$

Set (key k, value v) The operation Set saves the value v under the key k in the iterable mapping. The value v will now be retrievable with the key k and none of the existing keys or values in the iterable mapping are altered by the operation. Using the set operation on a key that is already mapped will simply overwrite the current mapping with the new one.

Postconditions:

- $\text{old}(IMap.\text{ContainsKey}(k)) \rightarrow IMap.\text{keys.length} = \text{old}(IMap.\text{keys.length})$
- $\overline{\text{old}(IMap.\text{ContainsKey}(k))} \rightarrow IMap.\text{keys.length} = \text{old}(IMap.\text{keys.length}) + 1$
- $IMap[k] = v$
- $\forall key.(key \neq k \rightarrow IMap[key] = \text{old}(IMap[key]))$
- $\forall key.(key \neq k \rightarrow (\text{old}(IMap.\text{ContainsKey}(key)) \leftrightarrow IMap.\text{ContainsKey}(key)))$
- $IMap.\text{ContainsKey}(k)$

Remove (key k) Remove removes the key k from the mapping. The key k will no longer be a valid key, and the item previously associated with it will no longer be retrievable through k. All other keys and their mapped values stay unchanged. The operation returns True if the key k was a valid key in the iterable mapping and False otherwise. If the operation returns False there was no change at all to the iterable mapping.

Postconditions:

- $\text{old}(IMap.\text{ContainsKey}(k)) \leftrightarrow \backslash\text{result} = \text{True}$
- $\backslash\text{result} = \text{False} \rightarrow IMap = \text{old}(IMap)$
- $\overline{IMap.\text{ContainsKey}(k)}$
- $\forall key.(key \neq k \rightarrow IMap[key] = \text{old}(IMap[key]))$
- $\forall key.(key \neq k \rightarrow \text{old}(IMap.\text{ContainsKey}(key)) \leftrightarrow IMap.\text{ContainsKey}(key))$

3.4 Caller Specific Mappings

A caller specific mapping is a mapping with integrated access control. This means it functions like a regular mapping, but places constraints on how it can be mutated by certain users. In our specific case, every caller should only be able to access the entry in the mapping that coincides with their address. This is possible because we are working with Solidity and so primitives such as the initiator of a certain method call are made available through the environment. This also necessitates that the key type of the mapping is the address type from Solidity. The value types of the caller specific mapping are restricted to those that would also be accepted by the Solidity compiler for a regular mapping. Just like with a regular mapping, values can be mapped to keys, however with the limitations mentioned above. Beyond accessing only the entries that belong to the caller, accessing and changing the entry of other callers is also possible, but only in a specified direction. This can be understood as a deposit functionality. Naturally, this only applies to certain value types such as uint or int, but extensions or alternatives could be implemented for other value data types.

Equality of caller specific mappings We consider two caller specific mappings A and B to be equal if they map the same values to the same keys, more specifically A maps a key to a specific value if and only if B also maps the same key to the same value.

3.4.1 Caller Specific Mapping Operations

For the following subsection *CSM* denotes an instantiation of our caller specific mapping data structure. *CSM*[k] refers to the value that is currently mapped to the key k. \caller refers to the address from which an operation was called.

Constructor The constructor returns a new instantiation of a caller specific mapping structure that is empty.

Get (address a) Get returns the value that is currently mapped to the key *a*. No constraints are placed on this operation in terms of access control, any caller could see the value of another caller's entry. The operation does not change the data structure in any way.

Postconditions:

- $CSM = \text{old}\,(CSM)$
- $\backslash\text{returns} = CSM[a]$

Get Get returns the value that is currently mapped to by the key of the caller. The operation does not change the data structure in any way.

Postconditions:

- $CSM = \text{old}\,(CSM)$
- $\backslash\text{returns} = CSM[\backslash\text{caller}]$

Set (value v) Set maps the value v to the caller's key in the CSM. Besides this, no changes are made to the CSM.

Postconditions:

- $CSM[\backslash\text{caller}] = v$
- $\forall address.(address \neq \backslash\text{caller} \rightarrow CSM[address] = \text{old}(CMS[address])$

Deposit (address adr, amount amt) Deposit increases the target's value in the CSM by the amount *amt*. The caller's entry in the CSM is reduced by an equal amount. False is returned if the caller does not have enough balance, the target's address is the same as the caller's or the amount is less than zero. All other entries outside the target's and the caller's remain unchanged.

Postconditions:

- $(amt < 0 \lor adr = \backslash\text{caller} \lor CSM[\backslash\text{caller}] < amt) \rightarrow \backslash\text{result} = \text{False}$
- $\forall address.(address \neq adr \land address \neq \backslash caller \rightarrow CSM[address] = \text{old}(CMS[address])$
- $\backslash\text{result} = \text{True} \rightarrow CSM[adr] = \text{old}(CSM[adr]) + amt$
- $\backslash\text{result} = \text{True} \rightarrow CSM[\backslash\text{caller}] = \text{old}(CSM[\backslash\text{caller}]) - amt$
- $\backslash\text{result} = \text{False} \rightarrow \forall address.(CSM[address] = \text{old}(CSM[address]))$

3.5 Iterator

An iterator is a data structure that offers simple operations to iterate over the elements of another data structure. For our purposes, an iterator can be seen as a support data structure for the other data structures of this chapter. The iterator does not provide functions to alter the underlying data structure that it iterates over.

3.5.1 Iterator Operations

For the following subsection *It* denotes an instantiation of our Iterator data structure. *It*[i] refers to a value of the underlying data structure that the iterator has assigned the index i to, independently of whether the underlying data structure indexes its elements. Iterator.index refers to the element the iterator is currently pointing to. Iterator.size refers to the total amount of elements the iterator is holding.

Constructor (data structure ds) The constructor returns a new instantiation of an iterator which is initialised with the collection of elements held by the data structure *ds*. The elements of *ds* are the ones which will be iterated over by the iterator.

HasNext The HasNext operation returns True if there is an element that hasn't been traversed yet. The HasNext operation doesn't alter the iterator in any way.

Postconditions:

- \returns = True $\leftrightarrow$ *It*.index + 1 < *It*.size

Next Next tells the iterator to jump to the next element in the collection and return it. If there is no next element, meaning we have already traversed all the elements in the collection an exception is thrown.

Postconditions:

- *It*.index = old(*It*.index) + 1
- \returns = *It*[*It*.index] + 1

4 Implementation in Solidity

This chapter is divided into three main sections, General, Challenges and Usage. The General section details the overall approach to and workflow of the implementation of the data structures in Solidity 0.7 and their specification and subsequent verification. In the Challenges section, we detail specific problems that were encountered and how they affected the implementation and specification. Lastly, the Usage section presents and describes how the outcome of the implementation can be used in future Solidity source code.

4.1 General

4.1.1 Approach

Implementation The overall approach to the matter was similar for all four data structures. First, a naive implementation for a specific data type in the form of a Solidity contract would be made. For our purposes at this stage, the Solidity contract was simply a definition of an object, with attributes and methods. The contract format allowed for easy initial testing and debugging of the implementation with the Remix IDE.[1] The Remix IDE allows for streamlined smart contract development and testing as it offers the possibility to deploy the contracts to a JavaScript VM Blockchain that runs completely in the browser.[2] Once deployed the methods defined on the contract can be called by the user and we can determine if the state changes they make align with what we expected.

Specification This approach led to data structures implemented in the form of smart contracts, which already mostly behaved per the formal definitions. The next big step was adding the formal specification. To use the solc-verify tool, the methods in the contract's source code have to be supplemented with comments that the tool can parse. The specific syntax of solc-verify is listed on the project's GitHub page (*solc-verify Documentation* n.d.).

At this point to aid in the writing of the functional specification we can use the pre- and postconditions from the formal specification. However, for the most part, we can't simply use these pre- and postconditions directly because the concrete implementations have further details that we have to respect, and the syntax of the solc-verify tool is also limited.

A common postcondition in the formal specification is that the data structure is the same before and after the operation, for example, $List = \text{old}(List)$. When formalising this condition with solc-verify we have to account for the fact that the list uses an array

[1] `https://remix.ethereum.org`

[2] `https://remix-ide.readthedocs.io/en/latest/create_deploy.html#deploy-the-contract`

internally to store the elements. To express that the list is the same before and after the operation, we specify that the array holding the elements is the same before and after the operation. To specify that two arrays are equal we can use either the explicit solc-verify array equality operator or iterate over both arrays and check each element for equality. In the formal specification such implementation details are simply abstracted. As a result, we have to adapt and extend the pre- and postconditions from the formal specification to fit our concrete implementations.

After the specification was written for the solc-verify tool, it can be used to detect inconsistencies between the method and its specification. If the verification fails, it is an indication that either the specification or method itself needs to be adjusted. If the tool reports a success the data structure can be considered functionally verified. In this case, the method and its specification are consistent with each other. More specifically under the assumption of the preconditions, after the method terminates the postconditions will be true. These guarantees are made under the assumption that solc-verify itself is correct.

Library conversion The next step was converting the contract to a library, why this was done is explained in subsection 4.2.2. This step involved only minor adjustments to the code and specification itself, furthermore, the results from the verifier weren't changed by this step. Lastly, the single verified library per data structure (of a specific data type, usually int) would be used to create a base document. From the base document, new libraries could be generated of the data structure, just with different types.

In essence, this was just the original library with all the references to the concrete data type replaced by tokens. Then a simple Python script was used to fill in the tokens, to generate libraries for the wanted basic Solidity types (uint, int, address, bool). The base libraries for each data structure can be found in the GitHub repository, they are the Solidity files with no type prefix. The used Python script can be found in the _Experimental Folder.

This completes the basic implementations, and we can now move on to more complex data types. For the complex types, we use the progress attained from the basic data types and attempt to modify the source code minimally to achieve our goal. For an explanation of the approaches and outcomes for complex data types reference the subsections 4.2.1 and 4.2.4.5.

4.1.2 solc-verify

Solc-verify (Hajdu and Jovanović, 2020) is an extended form of the Solidity compiler that enables source code verification. Specifications for methods are written in source code, solc-verify then checks for inconsistencies between the specification and the underlying code. To do this, solc-verify translates the source code into the Boogie programming language (Barnett et al., 2006) and uses the specification and existing boogie verification processes to generate contradictions or proofs.

In Figure 4.1 we see a source code example of a function `contains` that checks a contract local array to see if it contains a certain value. From the example we can see that the specification language supports postconditions, loop invariants and quantifiers (it also supports preconditions, state changes (*modifies*), contract invariants and more). Figure 4.1 additionally illustrates that the specification language is understandable and intuitive

regarding solidity language constructs, but in some cases is difficult to read and logically comprehend. One of the reasons for this is the lack of an explicit implication operator. Furthermore, statements that contain quantifiers must be written in the prenex normal form, which can make them difficult to read.

Other tools exist beyond solc-verify that can be used to add specifications to Solidity contracts and have them automatically verified. Some are quite close to the functionality of solc-verify, such as Verisol (Lahiri et al., 2018), but the specification language of Verisol does not support quantifiers, which makes it unsuitable for our use case. Celestial (Dharanikota et al., 2020) also offers functionality similar to that of solc-verify but does not support loop-invariants, which would be a considerable problem for the iterable data structures.

Besides the tools that offer a rich specification language, other tools that offer a simpler functionality exist, namely verification using constructs such as *require* and *assert* or similar. Regarding this, different approaches are used by the tools, but the functionality is ultimately similar for the user. The Solidity compiler itself, version 0.8.4 and onwards, features a built-in SMT checker (Alt and Reitwiessner, 2018) that enables specifying assertions and assumptions directly in the Solidity source code and will then issue warnings or error messages at compile time. ESBMC-Solidity (Song et al., 2022) and Solidifier (Antonino and Roscoe, 2021) are similar and offer a simple syntax in terms of specification. However, for our use case, we require a tool that supports quantifiers. Without quantifiers, it is impossible to express many of the properties we wish to verify. Hence solc-verify was chosen.

```
/// @notice postcondition exists (uint i) (!ret || (0 <= i && i < array.length &&
↪  array[i] == num))
/// @notice postcondition forall (uint i) (ret || !(0 <= i && i < array.length) ||
↪  array[i] != num
function contains(int256 num) public view returns (bool ret) {

    /// @notice invariant i >= 0 && i <= array.length
    /// @notice invariant i == 0 || array[i-1] != num
    /// @notice invariant forall (uint k) !(k >= 0 && k < i) || (array[k] != num)

    for (uint256 i = 0; i < array.length; i++) {
        if (array[i] == num) {
            return true;
        }
    }
    return false;
}
```

Figure 4.1: The contains method of a Solidity list smart contract and its specification.

4.2 Challenges

4.2.1 Generics in Solidity

As mentioned in subsection 4.1.1, for each data structure and basic data type, one library implementation was created. At this point, it would have been desirable if we could have created just a single library with a generic data type for each data structure and verified this. However, generics are not something the Solidity language supports currently. The language may support it in the future, but this topic is contended and not a top priority.[3] Despite this limitation multiple approaches to achieve some semblance of generics were pursued, unfortunately, none of them produced any meaningful results.

Workaround 1: Bytes Data Type The first approach that was explored to work around the restrictions, was to simply implement the data structures with raw bytes as a type since Solidity offers both fixed length and variable length bytes as primitive data types. However, to make this usable it would also require serialisation and deserialisation functions to turn the user-defined data types into a byte representation and back. The issue is that the user would have to implement these functions themselves, which would produce unnecessary overhead and complexity to the user's code.

In addition to that the solc-verify specification does not support the fixed length byte data type let alone the variable length version. The second restriction is however understandable as variable length bytes in solidity are just arrays of bytes1 (fixed size bytes of length one). This fact would have also made the data structure implementation using variable length bytes extremely inefficient as comparing two variable length bytes for equality would require iterating completely over two arrays every time. The fixed length bytes have an equality operator, however, the maximum length of a fixed size bytes type is 32 bytes, which means one byte32 could store at most a uint256 or int256. A struct consisting of only a uint/int-256 or less is rather unlikely, implying that fixed-size bytes would be unusable for a robust implementation in this sense.

Lastly, we also have the issue that since variable-length bytes are in reality just variable-length arrays of bytes, we run into issues when trying to use these as return values for functions in conjunction with the memory keyword, which is something that would be crucial to the implementation. In light of all these factors no such implementation using raw bytes was pursued.

Workaround 2: Structs as Generic Objects Another possible solution explored was using structs (with some restrictions) to emulate generic objects. The central issue with doing this (besides using structs as return values and parameters with the memory keyword in Solidity 0.7) is that the equality operator in Solidity is not defined for structs. Furthermore, we cannot define it and defining an explicit equality function that takes two structs as parameters and returns a bool value depending on the state of the given parameters is also not a solution. Such an equality method would work fine for the implementation, however,

[3] `https://github.com/ethereum/solidity/issues/869`

the solc-verify specification does not support user-defined functions, so we could not use it for the verification.

One might think a simple alternative to using an explicit equality function in the specification would be to just use the __verifier _eq construct that compares two structs for equality. However, because of how the __verifier _eq construct works, it would be of no use in this context, due to the different locations of the objects it would be comparing and its semantics (see A.1 for a more detailed explanation). Even if we could use the special solc-verify equality construct we would still be left with the problem that the old construct does not work with struct types.

The next idea would be to use the observation from above, that not having user-defined functions in the specification is not a real limitation. This means everywhere in the specification where we would have liked to use the equality function, we can just replace it with an equivalent specification that reflects the implementation of the equality function. This also means that specification would have to be generated dynamically, depending on the struct a user defines for their use case.

Such an approach raises several questions, primarily we must question if this type of implementation would even be in the interest of the user. Auto-generating the specification means we must either make some assumption about the behaviour the user wants or enable the user to parameterise the auto-generation in some sense. The user may consider two structs equal when a certain key variable is equal (such as an ID field or something similar), or maybe two structs are considered equal when all their fields are also equal. How the data structure will be used later is not something we would know during the implementation of the data structure itself. Any solution that adequately addresses this problem, by presenting a simple mechanism to auto-generate solc-verify specification based on the intended behaviour by a user, would introduce its own set of challenges and was deemed beyond the scope of this thesis.

To not leave this aspect of the data structures entirely unanswered, two list implementations were created where a user-defined struct is used as a data type. The first one is based on a complete user-definable struct with no limitations and an explicit `equals` method. The second is based on a user-defined struct with the limitation that the struct has an ID field of type uint. The ID field is used as a replacement for not having an `equals` method, as equality of instantiations of the struct is simply determined by the ID field. The implementations require at least Solidity version 0.8 or greater. Solidity 0.8 features more options concerning how structs can be used as parameters and return values which are crucial for a useful and intuitive implementation.

The first implementation is closest to what could be described as truly generic types (at least in the confines of the Solidity programming language), it does however not have a specification in light of all the problems described above. To use this data structure with a custom struct, the user would have to define the struct with the variables needed for their application and then also provide an implementation for the `equals` method. The second implementation features full specification and verification, under the caveat that equality comparisons are delegated to the comparison of the ID field. This presents a considerable limitation as the other fields the user may define for the struct are not featured in the specification at all. To use this version, the user would once again have to define the struct with the variables they require, but they would additionally have to incorporate the ID

field into the scope of their domain correctly. Overall, neither solution is entirely satisfying as they do not address the full scope of the underlying issue, however, the implementations are reasonable considering the many restrictions.

4.2.2 Libraries and Contracts in Solidity

When implementing structures in Solidity that aren't meant to be Smart Contracts, but rather used as constructs for future contracts, the most ideal variant is to implement them in the form of libraries. Libraries are rather simple and can be seen as a collection of static methods. Libraries cannot hold any state variables. All state is held by an actual contract that uses the library, by initialising structures defined by the library and using the library's functions on those structures to perform operations. In terms of classic object-oriented thinking, we do not have an object that holds an internal state and offers methods to query and change that state, rather the library defines what type of object we need to create as the user, and then offers methods that can be used on the object.

Using contracts to define new data structures is not a useful option as deploying a contract every time one wishes to use a specific data structure is as cumbersome to use as it is expensive and inefficient. While libraries offer a clean way to implement new primitives and functions they come with two main drawbacks that we have to accept in this case. They are explained in the following two paragraphs.

Limited Access Control It has been mentioned that a contract which wants to use the functionality of one of our libraries must initialise a structure defined in the library. Specifically, it initialises a struct. (These structs are similar to those found in the C family of programming languages.) This, however, leads to the issue that implementation details of our data structure are apparent to the contract and since the struct is initialised in the contract the programmer has access to all of its variables. In our case, the struct and the methods provided by the library implement an abstract data structure which ideally is only accessed through the methods defined in the library as these are functionally verified. If one changes the state of the struct directly, invariants that are assumed to hold inside the library could be broken, and therefore the data structure could fail to behave as expected. It is important to never modify the internals of the struct directly and to only use the methods provided by the Library to do so.

Solc-verify with Libraries Constructs from the solc-verify specification language such as invariants are not applicable to libraries as they do not hold any internal state. Nevertheless, we still wish to have invariants for our libraries. This limitation is not an issue as the statements that would have been invariants can simply be added to every method as post- and preconditions, achieving the same effect as an invariant. However while the methods themselves might all adhere to the invariants, one cannot guarantee that the user does not change the struct directly as mentioned above. At this point, we have to make the assumption that a possible use only alters the state of the struct through the designated methods.

While not presenting a massive practical issue, specifying invariants as pre- and postconditions for every method makes the specification overall more cluttered and less readable. Furthermore, we cannot use the *modifies* construct from the solc-verify specification language as again it cannot be applied to libraries since they do not hold any internal state. Fortunately, this is not an issue as any meaning the modifies statement has can be modelled with other parts of the specification. For example, if we want to make sure a certain variable does not change its value, we can include a postcondition which specifies that the variable is identical before and after the method was executed. Likewise, if we want to express that only a certain entry in an array is modified and the rest are unchanged we can use a universal quantification over the array and specify conditions for each entry.

4.2.3 The Iterator Design

Coming up with a useful implementation for an iterator in Solidity came with its own set of challenges. Naturally, the issue of deciding between a library and a contract also applies to the iterators, however, this has been discussed sufficiently in subsection 4.2.2 and will not be repeated here. A different substantial issue was that with the version of Solidity primarily used (0.7), using structs as return values for functions is not possible. Structs in return types of public and external functions may only have the calldata or storage modifiers, this presents an issue as creating a new instantiation of a struct in a function and passing it out is not possible. This happens because when creating an object in the scope of a function it will be allocated as a memory object and cannot be returned as a storage or calldata object. The solution would be to simply define the functions with memory keyword for the return value, but this is precisely what is not possible in Solidity 0.7. Therefore the natural way of implementing an iterator by giving the corresponding data structure a method to return it was not possible (with solc-verify 0.7).

The options that remained were to (a) implement the iterator methods directly in the data structure or (b) make the iterator a separate data structure that has to be explicitly initialised. Neither of the options is ideal as they both make the process of creating an iterator one can work with slightly more difficult. In the case of (a) the data structure would have to include a reset function which can be used to set the iterator back to the beginning. This presents two problems as the user must remember to call reset every time, they wish to iterate the structure anew but could also lead to issues with concurrent modification if the implementation does not split the elements of the iterator and the elements of the data structure cleanly.

In the case of (b) if we want to use the iterator we have to initialise an explicit iterator object for a certain data structure. After creating it we must then call another function to pass the data structure we want to iterate to the iterator. With this implementation, the iterator and data structure are separate objects (which was not the case in (a)). Ultimately it was decided to implement the second variant as it kept the concept of an iterator and the data structure it iterates apart, confining their concerns to individual libraries.

It should be mentioned, that a version of the solc-verify tool based on the Soldity 0.8.1 compiler exists, with which using structs as return values is possible. To this end, an implementation of an int set with an iterator that no longer requires the init workaround can be found in the _Experimental folder. This approach is however not standard practice

for the other data structures and types as solc-verify 0.8 is not yet stable.[4] Furthermore, the function that returns an instantiation of the iterator causes the verifier to crash with a multitude of error messages. To avoid any possible issues with the tool, especially regarding the formal verification the version based on solidity 0.7 was used as a standard. The implementations based on the 0.8 version should be seen as a proof of concept.

4.2.4 Deviations of the Concrete Implementations from the Abstract Definitions

The abstract data structures from chapter 3 are an idealised version of the data structures we want to implement. They are not constrained by any limitations of syntax and serve as the template to derive concrete implementations. The concrete implementations have some deviations from the abstract specification, in most cases due to necessity, they are documented and explained in the following section.

4.2.4.1 General Differences

Require and Preconditions The abstract definitions make use of preconditions to express certain criteria that must be met (for example by the parameters) for the method to function as expected. While solc-verify also supports preconditions, the choice was made to use Solidity's inbuilt require function instead.

The `require` function can evaluate a boolean expression, if the expression evaluates to false an exception is thrown, otherwise the control flow proceeds to the next line of code. Throwing an exception in Solidity means the method does not execute further and any state the method might have altered, before reaching the line where the exception was thrown, is reverted. This elegantly at execution time, ensures that preconditions to methods are met. Should the preconditions not be met the method does nothing and reverts.

Solc-verify reads `require` statements at the start of methods and implicitly adds them as preconditions to the specification of the method. This behaviour is advantageous as it acts as both specification and error handling in one. If for example, the `require` logic of the code was moved exclusively to the specification, we might still be able to functionally verify our methods, however, then it becomes the user's responsibility to ensure the preconditions are met. That is of course not ideal, thus the specification for all the methods have no preconditions besides the ones that are implicitly made through the require statements. Thus, it is ensured that if the method is called successfully (that is it does not revert), the postconditions will always be true.

Abstract and Concrete Specification Another aspect where differences are observable is in the specification of the concrete implementations. The concrete implementations usually have more pre- and postconditions than the formal specifications have. This happens partly because we have moved invariants to the functions as a result of not having contract invariants when using libraries (see 4.2.2 for more details). Another reason for this is that

[4] `https://github.com/SRI-CSL/solidity/issues/172`

formulating a statement abstractly is often more concise than the corresponding statement in solc-verify.

In some situations, however, we omit certain postconditions and enforce them instead with the Solidity view keyword. In Solidity, beyond the traditional access modifiers such as `public` and `private`, additional keywords can be included in the declaration of a function. The keyword `view` is of particular note, as it specifies that a function does not alter any state variables. This means for functions marked with `view`, specifying postconditions which ensure that the state before and after the function call is the same, are therefore not needed. This cleans up the specification slightly as we can delegate some of our specification work to the Solidity compiler, which will cause a compilation error if it detects behaviour in a function body that is inconsistent with its declaration.

Naming A third more minor detail that applies to all implementations is that the names of certain functions deviate from the ones specified in the formal definitions. This was necessary because the solc-verify solver will skip over the overloads of a function. Therefore, in the case of the lists, the Remove(index i) and Remove(element e) had to be split up into `remove(...)` and `removeElement(...)`. For sets, no changes had to be made. For iterable mappings, no changes had to be made. For caller specific mappings, Get() and Get(address a) were split up into `get()` and `getTarget(...)`.

4.2.4.2 The List IndexOf Implementation

A traditional IndexOf function for an array or list might return the index of an element if found or, -1 if not present. The abstract IndexOf definition conforms to this pattern. In Solidity, this behaviour immediately presents us with an issue because the array access operation denoted by the square brackets ([ ]), expects an unsigned int as an argument. This means we have two choices for the `IndexOf` implementation, use a regular int for the return type and thereby force a user to cast to uint every time they wish to use the index from `IndexOf` for element access. Alternatively, use a uint as the return type, this comes with the caveat that we can no longer use -1 to express that an element is not present in the collection.

Ultimately the choice was made in favour of the second option, namely to use a tuple output type as a replacement for not being able to return -1. This choice was made, as encouraging a user to cast was considered a worse alternative than a slightly unconventional return type. Technically the cast from uint to int with the return value from the `IndexOf` method should always work as expected as the maximum size of an array in Solidity is limited to $2^{64} - 1$.[5] Furthermore, arrays even close to this magnitude would incur massive gas costs meaning such a scenario would essentially be entirely unimaginable and unrealistic but not forcing a cast was decided upon to be the more elegant solution.

4.2.4.3 The List RemoveObject function

As was the case with the `IndexOf` function, the `RemoveObject` function for lists also has a tuple output type. The reasoning for needing it is slightly different. The formal specification

[5] `https://blog.soliditylang.org/2020/04/06/memory-creation-overflow-bug/`

of the remove element function dictates that all elements, besides the removed one, must be unchanged. Since we remove one element, the list becomes shorter and some of the indices are adjusted. Specifically, the elements that came after the removed element retain their order, but their index is decreased by one. The elements that came before the removed element are unchanged and retain their index.

In the statements which capture that the other elements are unchanged, the specification has to account for this index shift. It does this by using the index of the removed element as a boundary beyond which the indices of the elements have been reduced by one in comparison to before the operation started. Thus the index of the removed element has become an important part of the specification despite being neither a return type nor a parameter of the function. To access it in solc-verify the only option is to introduce it as an extra return value. This decision means the return type of the `RemoveObject` function becomes a tuple instead of a single bool. The first item of the tuple is a bool and can be seen as the regular output of the function as dictated by the formal definitions. The second item of the tuple specifies at which index an element was removed and is solely there because it needs to be used in the specification.

4.2.4.4 The Iterator Init Function

As a result of the choices and observations from the section 4.2.3 (which are that iterators are explicit objects, but not queryable from the structure they iterate) we are now faced with the issue of how exactly an iterator object can be initialised with the elements of the data structure it should iterate. Implementing a constructor concept when working with data structures defined through libraries is difficult. Furthermore, defining data structures through libraries is a central decision that was made for this thesis and is not something that can be changed as no sensible alternative exists (see the section 4.2.2).

When working with data structures defined through libraries, to use the data structure the user must initialise a struct that is defined in the library. This struct can then be treated as an instantiation of the data structure. When initialising structs explicitly defining their contained member variables is possible. Should this not be done, which is assumed to be the case, the member variables are initialised with standard values, usually just zero bytes.[6] The initial instantiation of the struct acts as the constructor call of the data structure. Unfortunately, Solidity gives us no means to add any extra logic to the basic struct initialisation. We also cannot use explicit struct member variable initialisation for this use case as not all struct variables can be initialised explicitly. Moreover, we also do not wish to use this type of initialisation, as the room for error when initialising member variables explicitly is large and it requires knowledge about implementation details of the iterator and data structure.

In other words, we are stuck with a constructor concept that cannot take arbitrary parameters and bluntly initialises all member variables of the struct to default values. Therefore the idea of passing a specific data structure such as a list as a parameter in the constructor of the iterator is not possible, this must be done explicitly after the fact. That is why the iterator implementation features an extra function called Init. The Init function

[6] `https://docs.soliditylang.org/en/v0.8.17/types.html`

serves as a pseudo constructor, has one parameter, and does what the constructor was meant to do. It accepts the data structure that is to be iterated and sets the internal state of the iterator. Only after having called Init on the iterator and passing the appropriate data structure is the iterator usable, before that it is simply empty and contains no elements. With solc-verify 0.8 this entire issue would not arise, as we would be able to pass an iterator initialised correctly out of a function.

4.2.4.5 Typing

The abstract definitions of the data structures include mostly no restrictions on what type of objects they can hold, naturally this is not something we can replicate completely. The subsection 4.2.1 discussed what options one has to approximate the functionality of generics. However, no conclusive results were reached. For the most part, the implementations are restricted to some of the primitive types in Solidity: booleans, unsigned/signed integers and addresses. Strings are not included as despite Solidity's support for Strings as a datatype, they cannot be compared by the equality operator (==). Not being able to use the equality operator in the implementation raises problems that are similar to the ones described in the 4.2.1 section. An explicit equality function would have to be implemented (which would even be efficiently possible)[7] but this could not be used in the specification, presenting a substantial obstacle.

Arrays and mappings aren't included either for related reasons described in the paragraph below. Bytes are also not included as they are not supported by the solc-verify specification. Lastly, Enums are not possible either as the crucial *old* construct in solc-verify, which is used to express the state of a variable before the function is executed, does not support user-defined Enums as a type itself. For each type and data structure pairing, an implementation is provided in the GitHub repository. For the iterable mappings, implementations for each key value type pair are provided, and for Caller Specific Mappings only two implementations from `address => uint` and `address => int` were made as these are the only combinations which are sensible within the scope of what the data structure aims to achieve.

Using Arrays and Mappings as Data Types At this point, it should be said the solc-verify specification does feature explicit equality comparisons for arrays and mappings. Meanwhile, the Solidity language itself does not. An initial idea might be to implement an explicit equality function and then verify it to ensure it behaves identically to the inbuilt equality function of the solc-verify specification. This approach would however only make sense in the case of arrays, because natural solidity mappings are not iterable. The only way to implement an equality function for mappings would be to iterate over the entire key space and check the value of each key for equality, this would be incredibly inefficient (excluding trivial cases such as the bool key type) and be unusable.

Furthermore, there are other problems with using arrays and mappings themselves as data types. Primarily the mapping-based data structures, iterable mappings and caller specific mappings, are intended to mirror the functionality of the standard Solidity mappings.

[7] `https://ethereum.stackexchange.com/questions/30912/how-to-compare-strings-in-solidity`

For regular solidity mappings, using either mappings or arrays as key types is not possible, so this is also something we do not wish to support with our mapping implementations. In addition, the caller specifc mappings and iterable mappings implementations use mappings internally, therefore it would not even be possible to support mappings and arrays as keys, regardless of whether it should be a feature. The other data structures, sets and lists, use arrays internally for element storage and the set also uses a mapping in addition for indexing support. This means for the sets the same issues persist that the calle specific mapping and iterable mapping already have.

Thus, lists are currently the only adequate candidate for an implementation with complex types. However, some caveats remain. While the implementation itself is possible, we immediately run into further problems when trying to specify and verify. The problems arise from the semantics of solc-verify explicit equality construct (*__verifier_eq*). It enables the comparison of structs, mappings and arrays but only when the two objects to compare are of the same location, meaning they must both be defined as storage or memory. Unfortunately for all the methods taking a data structure element as a parameter this means, the equality construct cannot be used in the specification of these methods as the data structure itself is saved in storage but the parameter in memory. Any attempts to compare elements of the data structure with the parameter will not be possible in this way unless the data structure definition is altered. However, even altering the definition does not entirely solve the problem (see section A.1 in the appendix for an exact explanation).

In light of all these restrictions and problems, only an implementation of a list holding integer arrays is provided (it has no specification or verification). Lists of arrays of other primitive types would look identical, arrays of types that aren't supported face the same issues the base types already have. Using nested arrays, that is using arrays of arrays is possible from Solidity 0.8 and onwards. While such implementations are interesting from a theoretical point of view, we can question whether these provide meaningful practical use. Nevertheless, the implementations are also available in the GitHub repository within the Experimental folder. It should be said that their use is discouraged as they reach the territory of runtimes with quadratic time complexity. While runtime complexities aren't an explicit part of this thesis they are something one should be acutely aware of when programming in Solidity, in addition to the gas costs function calls can incur.

4.2.5 Functions in the Specification

Unfortunately in the solc-verify specification language, incorporating user-defined methods into the specification is not possible. This makes the specification slightly harder to read, as the statements must then consist of implementation details that could be otherwise abstracted behind a method. While this is unfortunate it does not present any real limitations in the expressiveness of the specification. For example, we cannot use a `get(...)` method in the specification of a method of the list data structure, however, we can use that the list data structure is based on an array and use the square bracket array element access operator (array[i]) instead.

4.3 Usage of the Data Structures in Solidity Source Code

The implemented data structures can be used in future Solidity code in multiple ways. The recommended method is to use Solidity's import keyword. The import keyword loads all the symbols of a specified file into the current scope, meaning the methods, variables and structures defined in the specified file can be used in the current file. This concept is the analogue to modularization in other programming languages.

After importing the file we can initialise an instance of our data structure as defined in the library we have imported. This step is done in line 6 of Figure 4.2. In line 5 we tell the compiler to allow the methods defined in `address-set.sol` to be used as extension methods for the structure `Sets.set`. For example if we want to call the `isEmpty()` method on our set `mySet`, we can simply type `mySet.isEmpty()` (line 8). Without line 5, to achieve the same result we would have to use `isEmpty(mySet)`. While this is not a huge difference, the first variant using the extension methods is more comfortable to use.

On line 6 we are initialising a struct as defined in the address_set.sol library. At this point, since none of the member variables of the struct are specified, they are all initialised to their default values. This behaviour is important as the invariants used in the functional verification assume the data structures start in a state where all their internals are set to their default values. Should this not be the case the data structure is being used in a context it was not functionally verified for and the guarantees made by the verification may not hold anymore. Creating an invalid state of the data structure is also possible by simply accessing the struct member variables directly and altering them.

Therefore, initialising data structures only as shown in Figure 4.2 and not attempting to set any of the member variables directly, is crucial. Luckily for iterable mappings, caller specific mappings and sets making the first mistake is not possible as initialising a natural mapping in a struct with an existing one is not allowed by the Solidity compiler. Furthermore, with the iterators we need to call the `Init` function after having created the struct, to set it to the correct internal state before use. With the other data structures, this is not the case and they can be used immediately after initialisation, as in the example. For more examples reference the _Examples folder of the GitHub repository.[8]

Import specifics The Solidity command line compiler only supports importing Solidity files that are on the local file system, meanwhile, the Remix IDE also supports importing from the web. Sources such as Node Package Manager (NPM), GitHub and Swarm are supported by Remix. This means when developing or testing in the Remix IDE the data structures could also be imported directly from the corresponding GitHub repository wherein they are found. While convenient, making use of this feature is not recommended as it could introduce vulnerabilities.

If changes are made to a file stored on GitHub which is imported into a contract, this change may not be immediately obvious to the developer of the contract. Should the developer recompile the contract, the functionality of the contract can change, even when there was no change to the contract code itself, as behaviour from the imported contract may have changed. Naturally, local files can also be modified, but in such a scenario the

[8] `https://github.com/Joensw/solidity-formal-specification/tree/main/_Examples`

developer would probably be explicitly aware of the changes made and the files would not be vulnerable to some outside party, as is the case for files imported directly from an online source. Issues of this nature would only persist during the development of the contract and cannot affect already deployed contracts. Nevertheless, we recommend only importing files which are on the host file system and by extension to download the source files of the data structures from the GitHub repository before using them.

```
import "./Sets/address_set.sol";

contract Example {

    using Sets for Sets.Set;
    Sets.Set private mySet;

    bool isEmpty = mySet.isEmpty();
    // ...
}
```

Figure 4.2: Example initialisation of a set.

5 Evaluation

This chapter contains an evaluation and reflection on the results from the previous chapter. It features three main components. The Limitations section highlights some of the important problems and imperfections with the produced results that have already been eluded to in the Challenges section of the previous chapter. The Examples section extends on the usage section of the previous chapter and gives insight into what context the data structures can be used in. Lastly, the Timing section is a brief look into some timing metrics of the verifier.

5.1 Limitations

The implementation and verification of the data structures (at least with the basic four Solidity types) was mostly a success. All data structures could be verified with all their corresponding data types, except the iterable mappings and lists. For the iterable mappings using an address data type as the value type of the mapping produced failures within the verifier. The verification of the remove method would either time out, or the postcondition of the `remove` method specifying that all untracked keys are also unset ($\forall i \in \text{uint}(\text{self.indexOf[i]} == 0 \rightarrow \text{self.values[i]} == defaultValue)$), comes back as unproven. This failure is unfortunate and rather strange because it worked for the other types, however, none of the attempted efforts produced a verifiable result. In the case of lists, the bool data type was the problematic one. The postcondition of the `contains` function of the bool list specifying that there is an element which is equal to the parameter in the list if True is returned ($\text{isContained} \rightarrow \exists i \in \text{uint}(0 \leq i \wedge i < \text{self.items.length} \wedge \text{self.items[i]} == val)$), would also be marked as unproven. For the `removeObject` and `indexOf` functions the postcondition specifying that the function returns True should there be an element in the list equal to the given parameter ($\text{success} \rightarrow \exists i \in \text{uint}(0 \leq i \wedge i < \text{old(self.items.length)} \wedge \text{self.items[i]} == val)$) was marked as unproven.

Then there is still the case of the complex data types. Mappings as data types were not even possible to implement, and for arrays, an implementation was possible but not with a specification. Using structs as a datatype suffered similar issues unless the structs are forced to have an ID field, through which we can delegate the problem of element equality. While this approach is verifiable, it is definitely not perfect.

Aside from the issues with the actual verification the implementations themselves also contain some imperfections. Primarily it is not ideal but also unavoidable that the user can directly access the struct member variables and alter them. This presents a problem when speaking of verifiable correct data structures, as they are only correct as long as the user alters them with the verified methods and not directly. Changing the data structure directly

can put it into an inconsistent state, and cause the assumptions made in the verification to not hold anymore. Initialising the data structures in the wrong way causes the same problem. Therefore, we expect that the user only uses the data structures as intended.

Regarding the topic of usage, the way the data structures can be used in Solidity code might seem strange for someone used to other programming languages. However, supplying data structures for use this way is standard and common practice, large Solidity libraries such as OpenZeppelin use the same style.[1] Lastly, another flaw is the solc-verify specification language itself, it is rather difficult to read and verbose. This makes it challenging to understand what properties the postconditions guarantee. While each of the points touched upon isn't a massive issue, addressing them and the nuances they present is important.

5.2 Timing the Verification

To get a general idea of how long the verification takes for the individual libraries we can use the Linux time command to time the call of the verifier.[2] The measurements were performed using Windows Subsystem for Linux 2 on a machine with an i7-6700k processor.[3] The verification of each library was run three times and the mean was used as a result. From the output of the time command, the real time was considered. The real time is the amount of time that has elapsed for the calling user of the command, sometimes also referred to as wall time.

From the results (A.2), we can observe that between the different data types of the data structures, there is mostly no significant difference in the amount of required time. Between the data structures themselves, larger time differences are observed, this is expected as they do vary quite substantially in lines of code and lines of specification. For instance, the caller specific mappings are processed quite rapidly, they also feature only a few lines of code and specification and are rather simple. Whereas the lists, sets and iterable mappings feature many more lines of code and specification and take accordingly longer. Overall, the time taken to verify a single library is manageable.

The measurements were taken with the help of a Python script which calls the verifier (through the time command) on the according source code files and writes the output to a separate text file titled with the date and time. It additionally writes the results of the verification process to a separate text file which is labelled in a similar fashion. This makes the timing process more comfortable and less labour-intensive as we do not have to call the verifier on each file explicitly and collate the results. The Python script automates these tasks with a single call. The script (timer.py) can be found in the _Tools folder of the GitHub repository.[4] The raw outputs used in the tables can also be found in the _Timings sub-folders.

[1] https://github.com/OpenZeppelin/openzeppelin-contracts/tree/master/contracts/utils/structs

[2] https://man7.org/linux/man-pages/man1/time.1.html

[3] https://learn.microsoft.com/en-us/windows/wsl/

[4] https://github.com/Joensw/solidity-formal-specification/tree/main/_Tools

5.3 Example Use Cases of the Data Structures

As referenced in the general usage section (4.3) we provide example implementations for each data structure to display their concrete syntax in initialisation and use. These implementations are also examples of use cases where the data structure could be useful. The advantages are explained in the following paragraphs. The implementations can be found in the _Examples folder of the GitHub repository.[5]

Lists The list is probably the simplest of the data structures and the one with the most general functionality, therefore condensing it into a sole meaningful use case was challenging. The main advantages offered are the `indexOf(...)`, `remove(...)`, `removeObject(...)` and `contains(...)` functions. These are the basic quality-of-life functions and are not offered by standard Solidity dynamic arrays. (Note that the example for lists does not use any of these functions as creating a contract that effectively highlighted the benefits of the list proved to be difficult).

Sets In the example contract for sets, the set data structure is used to keep a collection of admin addresses (users) that are granted special privileges in the context of the contract. There is a privileged operation with a `requires` statement in the first line of the method body. This statement ensures the control flow only progresses to the next line if the caller's address is contained in the admins set. There is also a normal operation with no such restriction that everyone can call. To manage the admins set, besides the two addresses that are added in the constructor of the contract, there are operations to remove oneself from the admins set and also to add a new address to the set. Both of these operations are limited to callers whose address is already in the set. Finally, there is an iterate function which lacks any real functionality but could for example be extended to pay all the admins some Ether or perform any other various task with the admin addresses. The advantage of the set, in this case, is that the contract developer needs not to worry about duplicates in the set and adding to and removing from the set happens with a single line of code. The set takes care of all the specifics.

Iterable Mappings The iterable mapping was used to keep track of user accounts in a simple contract. The contract has functions for users to see, set and wipe their balances. Since iterable mappings are similar to the functionality of a regular mapping (excluding syntax), this in itself is nothing special as the same could have been accomplished easily with a regular mapping. The key difference is in the `payAll(...)` function that uses iteration over all user accounts to deposit an amount to all the active accounts. Accomplishing this functionality with a regular mapping would have required storing additional information, the iterable mapping hides this complexity and offers a simple way to query all set keys.

Caller Specific Mappings The main use case for caller-specific mappings is when a mapping with strict access control is needed. In the example, like with the iterable mappings, we have a balance mapping that users can deposit and withdraw from. Through the use of

[5] `https://github.com/Joensw/solidity-formal-specification/tree/main/_Examples`

the caller specific mapping the contract developer does not have to include the mapping key in the call. The key is automatically extracted as the caller's address. This ensures accessing the balances of other users through the contract is not possible and another benefit is that access control does not have to be implemented explicitly by the developer.

6 Related Work

Implementations of various data structures in Solidity can be found in many places around the internet. The concepts explored in this thesis in terms of implementation are hardly novel. Iterable mappings (or similar), for instance, are likely one of the more discussed and implemented Solidity data structures, many such implementations exist.[1] Sets are not quite as popular, but there is also an abundance of resources on the matter.[2] The same is true for lists.[3]

The different implementations differ greatly in some cases but often share similarities, as is to be expected. Many different goals can be pursued while implementing a data structure, efficiency and minimal memory usage are quite crucial for usage in smart contracts. For proofs-of-concepts, readability and ease of use might move to the foreground. Also of note is the iterator concept used in some of the implementations, which is quite different to the one used in this thesis. While we relied on an explicit iterator type defined through structs, some of the implementations cleverly used a single uint wrapped as a separate type for iteration.

Beyond the data structures which were relevant for this thesis, many more exist. An important name in this regard is the OpenZeppelin library, which is a repository that provides basic smart contract components in the form of contracts and libraries for reuse and modification. Security and reliability of the provided code are primary concerns and realised through extensive review and community vetting. The popular SafeMath library was also developed by OpenZeppelin.[4] Its goal was to effectively deal with overflows by treating them as explicit errors, which would cause transactions to revert. However, since Solidity 0.8, checking for overflows itself is part of the language[5] and the extra use of the library is now largely not required anymore. The advantage of the OpenZeppelin implementations compared to the implementations from various sources on the internet is their higher degree of certainty in safety and security. Code from the internet should always be treated with due regard, especially in the smart contract domain, as malicious individuals could use this as another attack vector.

[1] https://docs.soliditylang.org/en/v0.8.19/types.html#iterable-mappings, https://solidity-by-example.org/app/iterable-mapping/, https://github.com/rayonprotocol-research/solidity-iterable-mapping, https://docs.openzeppelin.com/contracts/4.x/api/utils#EnumerableMap

[2] https://github.com/rob-Hitchens/UnorderedKeySet, https://programtheblockchain.com/posts/2018/06/03/storage-patterns-set/

[3] https://medium.com/coinmonks/linked-lists-in-solidity-cfd967af389b, https://github.com/ethereum/dapp-bin/blob/master/library/linkedList.sol, https://github.com/vittominacori/solidity-linked-list/blob/master/contracts/StructuredLinkedList.sol, https://gist.github.com/benzap/4adff79e702c595c5277d2ebcdf90877

[4] https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/math/SafeMath.sol

[5] https://docs.soliditylang.org/en/v0.8.0/080-breaking-changes.html

While the resources from OpenZeppelin and other sources provide much utility, the primary difference of this thesis is that we used explicit formal verification to guarantee behavioural aspects of the provided data structures. A further important distinction is the style of implementation, while OpenZeppelin uses the library style for data structures many of the implementations from other sources do not. Using a contract style for a proof-of-concept is sufficient, but implementing the data structure through a library is ideal for future usage. See subsection 4.2.2 for further details on this topic.

Another interesting aspect is how the implementations dealt with the lack of generics in Solidity, most were a single implementation and thus a single type. However, the enumerable set implementation from OpenZeppelin is implemented with the bytes32 datatype. From the bytes32 implementation, several implementations with concrete data types that fit into a bytes32 are derived using the base implementation by casting. This approach is different to the approach used in this thesis, as we generated the concrete data structures of different types from a base contract through text replacement. The approach using bytes32 is more elegant as it results in less duplicated code, details about why a different approach was chosen for this thesis can be found in subsection 4.2.1.

7 Conclusion

In this thesis, we formally defined and then introduced Solidity implementations of list, set, iterable mapping and caller specific mapping data structures. The implementations cover the basic types in Solidity (address, bool, uint, int). We functionally verified the implementations using the solc-verify tool. Most of the implementations could be successfully verified, however iterable mappings with the address type as a key and bool lists are the exceptions. Furthermore, approaches were explored to enable the data structures to store more generic elements in the form of Solidity structs, raw bytes, arrays and mappings. However, for these types limitations with the solc-verify tool and Solidity language itself presented insurmountable obstacles. In this regard unverified and partially verified implementations were produced.

The implementations of the data structures themselves are not without limitations, especially in terms of efficiency and usability. However, the successful functional verification not only provides secure data structures that could be used in future Solidity smart contracts but also demonstrates that functionally verifying data structures in Solidity is viable with the existing tools.

Future work It is entirely possible that there exists some workaround to the different data location equality comparison dilemma that prevented us from using the verifier equality construct. One approach that may have promise is introducing a local variable to act as a memory copy for specific entries in the data structure that are saved in storage. In this scenario using the verifier equals construct would then be possible. While this might be a potential approach, it would make the code considerably less readable and less efficient.

Another caveat that would have to be addressed is that the specification language can only reference either parameters, return values or contract/library level variables. Writing specification targeting the intermediate variables themselves is not possible. To work around this the intermediate variable may then have to be specified as a garbage output value just so it can be accessed by the specification. This approach will, however, break down when we need to do many equality comparisons in a single function (such as the case with contains). These types of situations may be avoided by keeping more information in the data structure, such as in the case of the contains call, another mapping that trivialises the contains function. Another consideration might then be to use a dynamic array as a garbage output value to keep track of all the copy variables introduced in the function. As is evident plenty of room for further investigation exists in this regard.

Another aspect that can be explored is using other verification tools to possibly address some of the issues encountered with solc-verify. Many of the existing tools besides solc-verify were initially deemed inadequate as they lacked certain key functionality (such as invariants or quantifiers). Nonetheless, it would be interesting to investigate if any of the

other tools address the equality aspect differently and what the implications of this for semi-generic Solidity data structures would be.

Finally, as the iterable mapping verification was not entirely successful and since the iterable mapping itself is one of the more useful data structures in the domain of smart contracts it might be worthwhile looking at its functional verification from a few more perspectives. As mentioned above other tools might offer some benefit in this regard. Also possible is that a different implementation of the data structure would be verifiable with solc-verify, an interesting candidate for this would certainly be the OpenZeppelin enumerable map implementation.[1]

[1] `https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/structs/EnumerableMap.sol`

Bibliography

Alt, Leonardo and Christian Reitwiessner (2018). "SMT-Based Verification of Solidity Smart Contracts". In: *Leveraging Applications of Formal Methods, Verification and Validation. Industrial Practice*. Ed. by Tiziana Margaria and Bernhard Steffen. Cham: Springer International Publishing, pp. 376–388. ISBN: 978-3-030-03427-6.

Antonino, Pedro and A. W. Roscoe (2021). "Solidifier: Bounded Model Checking Solidity Using Lazy Contract Deployment and Precise Memory Modelling". In: *Proceedings of the 36th Annual ACM Symposium on Applied Computing*. SAC '21. Virtual Event, Republic of Korea: Association for Computing Machinery, pp. 1788–1797. ISBN: 9781450381048. DOI: 10.1145/3412841.3442051. URL: https://doi.org/10.1145/3412841.3442051.

Atzei, Nicola, Massimo Bartoletti, and Tiziana Cimoli (2017). "A Survey of Attacks on Ethereum Smart Contracts SoK". In: *Proceedings of the 6th International Conference on Principles of Security and Trust - Volume 10204*. Berlin, Heidelberg: Springer-Verlag, pp. 164–186. ISBN: 9783662544549. DOI: 10.1007/978-3-662-54455-6_8. URL: https://doi.org/10.1007/978-3-662-54455-6_8.

Barnett, Mike et al. (2006). "Boogie: A Modular Reusable Verifier for Object-Oriented Programs". In: *Formal Methods for Components and Objects*. Ed. by Frank S. de Boer et al. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 364–387. ISBN: 978-3-540-36750-5.

Buterin, Vitalik (2014). "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform". In: URL: https://ethereum.org/669c9e2e2027310b6b3cdce6e1c52962/Ethereum_Whitepaper_-_Buterin_2014.pdf.

Dharanikota, Samvid et al. (Dec. 2020). *Celestial: A Smart Contracts Verification Framework*. Tech. rep. MSR-TR-2020-43. Microsoft. URL: https://www.microsoft.com/en-us/research/publication/celestial-a-smart-contracts-verification-framework/.

Hajdu, Ákos and Dejan Jovanović (2020). "solc-verify: A Modular Verifier for Solidity Smart Contracts". In: *Verified Software. Theories, Tools, and Experiments*. Ed. by Supratik Chakraborty and Jorge A. Navas. Cham: Springer International Publishing, pp. 161–179. ISBN: 978-3-030-41600-3.

Lahiri, Shuvendu K. et al. (2018). "Formal Specification and Verification of Smart Contracts for Azure Blockchain". In: *CoRR* abs/1812.08829. arXiv: 1812.08829. URL: http://arxiv.org/abs/1812.08829.

Mehar, Izhar et al. (Jan. 2019). "Understanding a Revolutionary and Flawed Grand Experiment in Blockchain: The DAO Attack". In: *Journal of Cases on Information Technology* 21, pp. 19–32. DOI: 10.4018/JCIT.2019010102.

Salzer, Gernot and Monika Di Angelo (Apr. 2019). "A Survey of Tools for Analyzing Ethereum Smart Contracts". In: DOI: 10.1109/DAPPCON.2019.00018.

Sayeed, Sarwar, Hector Marco-Gisbert, and Tom Caira (Jan. 2020). "Smart Contract: Attacks and Protections". In: *IEEE Access* PP, pp. 1–1. DOI: 10.1109/ACCESS.2020.2970495.

solc-verify Documentation (n.d.). https://github.com/SRI-CSL/solidity/blob/0.7/SOLC-VERIFY-README.md#solc-verify. Accessed on 12.5.2023.

Solidity Documentation (n.d.). https://docs.soliditylang.org/en/v0.8.20/#solidity. Accessed on 11.5.2023.

Song, Kunjian et al. (2022). "ESBMC-Solidity: An SMT-Based Model Checker for Solidity Smart Contracts". In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. ICSE '22. Pittsburgh, Pennsylvania: Association for Computing Machinery, pp. 65–69. ISBN: 9781450392235. DOI: 10.1145/3510454.3516855. URL: https://doi.org/10.1145/3510454.3516855.

Wohrer, Maximilian and Uwe Zdun (Mar. 2018). "Smart contracts: security patterns in the ethereum ecosystem and solidity". In: pp. 2–8. DOI: 10.1109/IWBOSE.2018.8327565.

A Appendix

A.1 Data Locations in Solidity

Variables in Solidity have two main types of data locations, memory and storage. In simple terms, memory refers to non-persistent storage, such as in the local scope of a method. Storage refers to persistent storage such as local variables of a contract. A variable will usually have a default data location, depending on the context it is in. Sometimes the storage location needs to be explicitly stated. In some cases, this default can be overwritten by explicitly stating the desired location. The default location for function parameters and return types is memory. The difference between the memory and storage keywords for function parameters is subtle but important.

If a function parameter is designated with memory in the definition, within the function we work on a copy of what the caller passed to the function, we cannot modify the original object the caller passed. This is essentially a call-by-value concept. Conversely, if a function parameter is designated with storage, modifying that parameter within the method body will also modify the object the caller passed into the function. A storage parameter is just a reference to a storage variable defined in a different context, no copying happens. This is similar to a call-by-reference concept.

Since the methods in the data structures are defined in a library but also need to be able to modify the data structure, there is no option but to pass the data structure itself as a parameter to the method with the storage modifier. This way the method can make changes to the data structure as it receives a reference to it. However, this also means the user can only initialise the data structures for use in storage, that is as a contract variable. This is because Solidity is strict when it comes to data locations, for example, the compiler will only allow a method defined with a storage parameter to be called with a variable that is actually in storage, and the method cannot be called with a variable that is in memory. There are some nuanced exceptions to this behaviour but they are not applicable to our use case.

Therefore, initialising a data structure for sole use in a single method call, which is then later discarded, is not possible. We do not consider this a significant issue as in any case such a scenario would be rather uncommon for the data structures. The decision of storing the data structures in storage becomes relevant again when trying to use solc-verify *__verifier_equals* construct as it requires the variables that it is comparing to be in the same data location. Our data structures are saved in storage out of necessity, the other parameters of the functions are saved in memory for convenience. Saving the parameters, other than the data structure itself, in storage is possible, but requires a few adjustments to the contract code. We do however not use this variant, as it does not aid us in our specification efforts.

While saving everything in storage would mean we can use the *__verifier_equals* construct in the specification, its semantics still get in the way. For storage types the *__verifier_equals* construct performs a reference equality check, and for memory types a deep equality check (*solc-verify Documentation* n.d.). A reference equality check is simply not the behaviour we need. The main use case for the construct would be comparing the parameters of functions with elements that are already stored in the data structure. In this case, the reference equality is pointless as we are not interested in if two objects are actually the same object, but rather if two different objects are identical. The deep equality check for the memory types would be a better fit in this scenario. However, as described in the paragraphs above, the fact the data structures are saved in storage is non-negotiable. Hence, the solc-verify *__verifier_equals* construct is not something we can use in our case.

A.2 Timing Tables

This section contains the tabular results of timing the verification process. The entries marked with * caused the verifier to fail, or timeout.

Data type	Time taken (s) (2 d.p.)
address	11.40
bool	11.31 *
int	11.30
uint	11.32

Table A.1: Length of verification duration for the list data structure of different data types in seconds.

Data type	Time taken (s) (2 d.p.)
address	11.46
bool	11.53
int	11.48
uint	11.53

Table A.2: Length of verification duration for the set data structure of different data types in seconds.

Data type	Time taken (s) (2 d.p.)
int	2.02
uint	1.87

Table A.3: Length of verification duration for the caller specific mapping data structure of different data types in seconds.

Data type	Time taken (s) (2 d.p.)
address → address	12.20 *
address→ bool	12.32
address → int	12.29
address → uint	12.17
bool → address	12.17 *
bool → bool	12.30
bool → int	12.18
bool → uint	12.31
int → address	12.20 *
int → bool	12.30
int → int	12.20
int → uint	12.27
uint → address	12.29 *
uint → bool	12.26
uint → int	12.33
uint → uint	12.13

Table A.4: Length of verification duration for the iterable mapping data structure of different data types in seconds.